Unit IV

VHDL

Very High speed Integrated Circuit (VHSIC) Hardware Description Language

Use of VHDL:
- To design digital systems before building them
- To simulate & test circuits
- To reduce error and costs

General structure of VHDL program
VHDL is made up of different blocks, main three parts: 1. Library section
2. Entity Declaration
3. Architecture Declaration

1. Library section
- Provides predefined data types and functions. (e.g. std_logic, std_logic_vector)

e.g.
```
library ieee;
use ieee.std_logic_1164.all;
```

2. Entity Section
- Define external view (black box) of system, specifies inputs and outputs

Syntax:
```
entity entity_name is
port (
    signal_name : mode type
);
```

Modes:
- in
- out
- inout
- buffer

3. Architecture section
- connected to entity, defines how system work internally.
- can be written in different forms: Behavioral, Dataflow, structural
- Two parts:
  1. Declaration part — Signals, constants, components
  2. Execution part — Logic statements

notes.arpanadhikari7.com.np

adhikari-arpan/BECE_Notes

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

```
architecture arch_name of entity_name is
    --declarations
begin
    --logic implementation (execution part)
end arch_name;
```

Optional parts:

4. Package

Package is a VHDL design unit used to declare common data types, constants, functions and procedures that can be shared across multiple VHDL modules.

- Reusability, used with "use" statement.

For e.g. use IEEE.std_logic_1164.all;

(Package)

(i) ~~Package declaration~~

```
package my_package is
    constant N : integer := 8;
end my_package;
```

(ii) Package body

```
library ieee;
use ieee.std_logic_1164.all;
use work.my_package.all;
std_logic_vector (N-1 downto 0);
```

5. Configuration

- Used to bind a entity to an architecture.
- Used when multiple architecture exist for one entity.

Modeling styles

1. Behavioral Modeling

- Describes what system does ,Highest Level Abstraction
- Uses process block
- written like a program (sequential statements)
- Simulation oriented allowing designers to validate functionality before moving to hardware implementation.

adhikari-arpan/BECE_Notes

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Concurrent statement → statement running parallely.

} Sequential statements → statement execute one after another

Combinational ckt → No memory
Sequential ckt → Memory used.

2. Dataflow Modeling
- Describes how data flows from i/p to o/p , Middle Level Abstraction.
- Uses concurrent statements.
- Based on boolean expressions, logical equations, simple relationships bet^n i/p & o/p.

3. Structural Modeling
- Describes how system is built using components (gates/modules), lowest level abstraction
- Focus : structure (hardware connection), closely reflects actual hardware, suitable for managing large & intricate designs
- similar to circuit diagram, Elements: component declaration, component instantiation, Port Mapping

Datatypes in VHDL

Datatypes define what kind of values a signal, variable or constant can hold.
Two main categories: 1. Scalar Type - single value
2. Composite Type - multiple value combined

1. Scalar Type

a) predefined scalar type: built-in provided by VHDL
- Eg. bit → '0', '1'
- boolean → TRUE, FALSE
- integer → whole numbers
- real → decimal numbers
- std_logic → multi-valued logic

e.g. signal a : std_logic;
signal count : integer;

b) User defined data type : User defines list of name values.
Used in Finite state machines, control system

e.g. type state is (IDLE, START, STOP);
signal s : state
s can take only one of these values.

notes.arpanadhikari7.com.np

Scanned with CamScanner
Available on notes.arpanadhikari7.com.np

2. Composite type → store multiple values.

a) Array Type: Collection of same type element
Used for data buses, memory.
E.g. bit_vector
std_logic_vector
signal data: std_logic_vector(7 downto 0);

b) Record Type: collection of different data types
Used for complex data structures.
E.g. type student is record
name: string(1 to 10);
age: integer;
end record;

Subprogram

Subprograms are reusable blocks of code used to perform specific task in VHDL design.
Benefits: Code readability, Modularity, reusability, Maintainability

notes.arpanadhikari7.com.np

Types:

1. Function
- Takes input, perform computation & return value.
- Must return one value.
- cannot modify signal directly.

e.g. function add (a,b: integer) return integer is
begin
return a+b;
end function;

Use: signal result: integer;
result <= add(5,3);

Scanned with CamScanner
Available on notes.arpanadhikari7.com.np

2 Procedure

- can take multiple input/output, donot return value directly
- can modify signal/variables. uses in, out, inout parameters.

```
procedure add-proc(
    signal a,b : in integer;
    signal result : out integer
) is
begin
    result <= a+b;
end procedure;

Use: add-proc(x,y,sum);
```

structural elements:

1. Component Declaration

Defines interface of component by specifying its input & output ports. Component declarations provide a blueprint for components to be used & are placed in architecture section or package.

```
component componentName is
    Port (
        Port1 : in STD_LOGIC;
        Port2 : in std_logic
            -- more ports
    );
end component;
```

notes.arpanadhikari7.com.np

PN

adhikari-arpan/BECE_Notes

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

2. Component Instantiation & port mapping

Each instance correspond to individual realization.

- Instantiation is used to create physical representations in design.
- Port mapping connects signal defined in architecture to port of components during instantiation.
- It ensures proper flow of data bet^n components.

```
syntax: InstanceLabel : ComponentName
        Port map(
            port1 => signal1,
            Port2 => signal2
            --Additional mapping
        );
```

4. Mixed Modeling

Combines 3 modeling styles. Allows designers to leverage strength of each style for diff. part of system. Highly practical method → flexibility, readability, scalability

Package

Package is a VHDL design unit used to declare common datatypes, constants, functions, procedures that can be shared across multiple VHDL modules.

Structure of package: 1. Package declaration
2. Package Body

1. Package declaration ; contains type definitions, constants, function/procedure declaration

```
package my_package is
    constant N : integer := 8;
    function add(a,b : integer) return integer;
end my_package;
```

adhikari-arpan/BECE_Notes

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

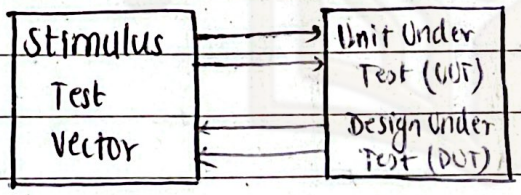
2. Package body : contains actual implementation of functions/procedure

```
package body my_package is
    function add (a,b : integer) return integer is
    begin
        return a+b;
    end function;
end my_package;
```

Using:
```
library IEEE;
use work.my_package.all;
```

Testbench

Testbench is a VHDL program used to verify the functionality of a design (UUT/DUT) by applying different input conditions (stimulus) and observing the output response.



Testbench Architecture

notes.arpanadhikari7.com.np

Components: 1. Entity Declaration
2. Architecture Declaration
3. Component Declaration
4. Signal Declaration
5. Clock and stimulus Generation
6. component Instantiation

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Behavioral ⇒ Process

if, elsif, else

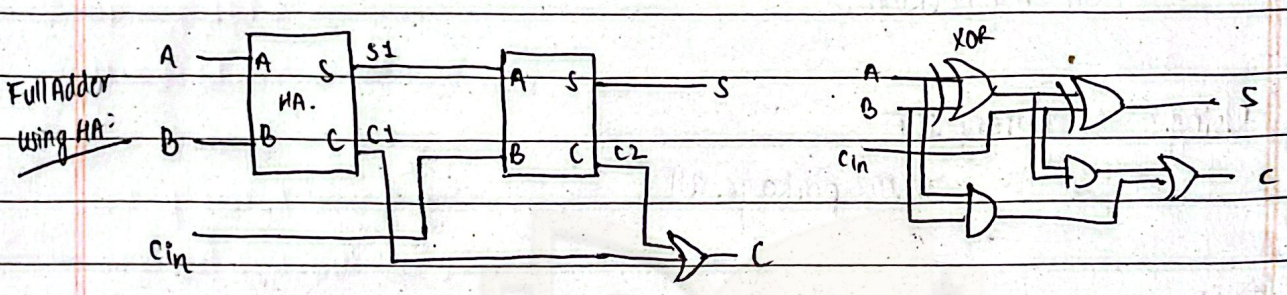
if ( ) then
- - - -
end if;

Universal gates : NAND, NOR
Special gates : XOR, XNOR

		XOR	XNOR
0	0	0	1
0	1	1	0
1	0	1	0
1	1	0	1

Adder

Sum = A xor B } A xor B xor C
Carry = A and B } (A and B) or (B and C) or (A and C)



Code for full adder using half adder as component :

VHDL code for HA :

```
library IEEE;
use IEEE.std_logic_1164.all;

entity half_adder is
 port (
      A, B : in std_logic;
      sum, carry : out std_logic
 );
 end half_adder

architecture behavioral of half_adder is
begin
   sum <= A xor B;
   carry <= A and B;
end behavioral;
```

notes.arpanadhikari7.com.np

Scanned with CamScanner
Available on notes.arpanadhikari7.com.np

component_name -> signal_name

VHDL code for ORGate:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity ORGate is
port (
    X,Y : in std_logic;
    Z : out std_logic
);
end ORGate;

architecture behavioral of ORGate is
begin
    Z <= X or Y;
end behavioral;
```

FA:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity FullAdder is
port (A,B,Cin : in std_logic;
      sum,carry : out std_logic
);
end FullAdder;

architecture structural of FullAdder is

component HalfAdder
port (
    A,B : in std_logic;
    sum,carry : out std_logic);
end component;

component ORGate
port (X,Y : in std_logic;
      Z : out std_logic);
end component;

signal S1, C1, C2 : std_logic;

begin
HA1 : HalfAdder port map
    (A => A, B => B, sum => S1, carry => C1);

HA2 : HalfAdder port map
(A => S1, B => Cin, sum => sum, carry => C2);

OR1 : ORGate port map
(X => C1, Y => C2, Z => carry);
end structural;
```

notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

~~Testbench for Full subtractor~~:

Subtractor

diff <= A xor B
borrow <= (not A) and B

diff <= A xor B xor C
borrow <= (not A and B) or (not A and C) or (B and C)

Testbench for FS:

A	B	Bin	diff	borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

```
library IEEE;
use IEEE.std_logic_1164.all;

entity test_full_sub is
~~end~~ end test_full_sub;

architecture arch_fs of test_full_sub is
  component full_sub
   port ( A, B, Bin : in std_logic;
          diff, borrow : out std_logic );
   end component;

   signal A, B, Bin ~~diff, borrow~~ : std_logic := '0';
   signal diff, borrow : std_logic;

   begin
     UUT : full_sub port map
          (A => A, B => B, Bin => Bin, diff => diff, borrow => borrow);

     stim_proc : process
      begin
          A <= '0'; B <= '0'; Bin <= '0'; wait for 100 ps; ----
     end process;
end arch_fs;
```

adhikari-arpan/BECE_Notes

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

case:
when " " => --
When others =>
end case;

Multiplexer

4:1	S1	S0	Y
	0	0	I(0)
	0	1	I(1)
	1	0	I(2)
	1	1	I(3)

Y <= (not S1 and not S0 and I(0))
or (not S1 and S0 and I(1) or ..

```
entity mux is
port ( I : in std_logic_vector (3 downto 0);   (Input)
       S : in std_logic_vector (1 downto 0);   (Select lines)
       Y : out std_logic );                    (Output)
end mux;
process(I,S) begin
case S is
  when "00" => Y <= I(0);
end case; others => Y <= '0';
```

#Testbench → Initialize value for I signal signal I : std_logic_vector (3 downto 0) := "0101";

De-Multiplexer

1:4	S1	S0	Active o/p
	0	0	Y(0)
	0	1	Y(1)
	1	0	Y(2)
	1	1	Y(3)

Y(0) <= I and not S1 and not S0
Y(1) <= I and not S1 and S0

```
entity demux is
  port (I : in std_logic;
        S : in std_logic_vector (1 downto 0);
        Y : out std_logic_vector(3 downto 0) );
end demux;

case S is
  when "00" => Y(0) <= I ;
    others => Y <= "0000".
```

Initialize I <= 1 inside process (stim_proc : process).

notes.arpanadhikari7.com.np

Encoder

D								A		
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0

A(0) <= D(1) or ..
A(1) <= D(2) or ..

```
port( D : in std_logic_vector (7 downto 0);
      A : out std_logic_vector(2 downto 0) );

when "00000001" => A <= "000";
```

Decoder

I				Y							
0	0	0		0	0	0	0	0	0	0	1
0	0	1		0	0	0	0	0	0	1	0

Y(0) <= not I(0) and not I(1) and not I(2)

```
I : in std_logic_vector(2 downto 0);
Y : out std_logic_vector (7 downto 0) );
case I is
  when "000" => Y <= "00000001";
```


Scanned with CamScanner
Available on notes.arpanadhikari7.com.np

Flip flops

S	R	Q(Next state)
0	0	No change
0	1	Reset (0)
1	0	Set (1)
1	1	Invalid

J	K	Q(Next State)
0	0	No change
0	1	Reset (0)
1	0	Set (1)
1	1	Toggle

D	Q(Next state)
0	0
1	1

If (clk = '1'), preQ <= D;

T	Q(Next state)
0	No change
1	Toggle

JK:

```
entity JK_ff is
port(
    J, K, clk : in std_logic;
    Q, Qn : out std_logic);
end JK_ff.

architecture behavioral of JK_ff is
 signal preQ : std_logic := '0';
begin
   if (clk = '1') then
   process (J, K, clk)
   begin
     if (clk = '1') then
       if (J = '0' and K = '0') then
           preQ <= preQ;
       elsif (J = '0' and K = '1') then
           preQ <= '0';
       elsif (J = '1' and K = '0') then
           preQ <= '1';
       else
           preQ <= not preQ;
       end if;
     end if;
   end process;

   Q <= preQ;
   Qn <= not preQ;

end behavioral;
```

adhikari-arpan/BECE_Notes

Scanned with CamScanner

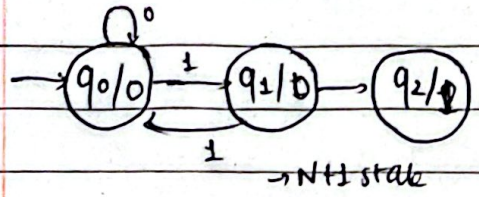
Available on notes.arpanadhikari7.com.np

Testbench (after port map, before stim_proc)

```
clk_process : process
begin
    clk <= '0'; wait for 0 ps;
    clk <= '1'; wait for 5 ps;
end process;
```

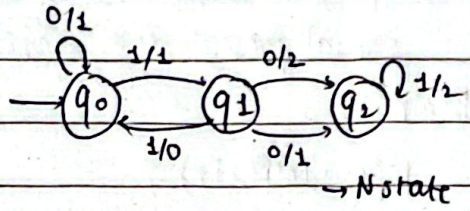
Moore Machine

Output depends only on present state.



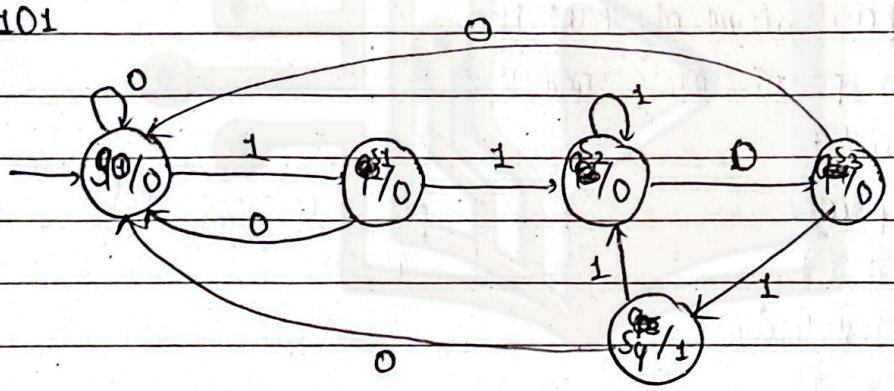
Mealy Machine

Output depends on present state and present input.



Sequence detector using Moore Machine

Q) Detect 1101



notes.arpanadhikari7.com.np

FSM : 3 blocks

- → process (clk, clr)
- → process (present_state, din)
- → process (present_state)

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

```
library IEEE;
use IEEE.std_logic_1164.all;

entity fsm is
 port ( clk, clr, din : in std_logic;
        dout          : out std_logic );
end fsm;

architecture fsm_arch of fsm is
   type state is (s0, s1, s2, s3, s4);
   signal present_state , next_state : state;
 begin
   process(clk,clr)
   begin
      if (clr = '1') then
         present_state <= s0;
      elsif rising_edge (clk) then
         present_state <= next_state;
      end if;
   end process;

process(present_state, din)
begin
 case present_state is
   when s0 => if din = '1' then
                 next_state <= s1;
              else
                 next_state <= s0;
              end if;
   when s1 => ---
      s2, s3, s4 - - -
   when others => next_state <= s0;
 end case;
end process;

process (present_state)
 begin
   if present_state = s4 then
      dout <= '1';
   else
      dout <= '0';
   end if;
 end process;

end fsm_arch;
```

adhikari-arpan/BECE_Notes

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Operators (Allows to perform various operations on data objects such as signals, variables, constants)

1. Logical operators
   (logical AND, logical OR, NAND, NOR, L.XOR, L.XNOR, L.NOT)
2. Relational operators
   (=, /= (Not equal to), <, >, <=, >=)
3. Shift Operators
   shift left logical (sll), srl, sla (shift left arithmetic), sra, rol (rotate left, ror
4. Arithmetic operators
   +, -, *, /, mod, rem
5. Concatenation operator
   & (eg. Vector <= A & B)
6. Miscellaneous operator
   ?: conditional operator Result <= (A>B) ? A : B

Unit-5

Wireless communication (communication achieved through electromagnetic waves

1. Bluetooth
   Range : 10-100 m
   Speed : upto 2 Mbps (BLE)
   Power Efficiency : Optimized for minimal energy usage.
   Network type : supports point-to-point or small network (called piconet)
   - Operates in 2.4 GHz ISM band

   Types: Bluetooth classic, Bluetooth low Energy (BLE)

   Applications : Wearables, Smart Home devices, Audio Devices

notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np